Week 4 Practice: Vue Components and Reactivity

Oct 13, 2026 · @thavorac chun

In 2 hours you build the Campus Events page from components, get feedback on your proposal (M0), and start your own Vue app with hard-coded data. Starter code: `week04-starter.zip`.

Part A: Run the starter and read the tests (10 min)

Campus Events is the reference app we build together in every practice session. This week it shows events from a hard-coded file (`src/data/events.ts`). Unzip the starter into `week04/` of your practice repository.

```
cd week04/campus-events-week04
npm install
npm run dev     # http://localhost:5173
npm test        # 15 tests, most fail for now
```

1. Open the page. Every card says "Git Workshop": the components show hard-coded text.
2. Open `tests/EventCard.test.ts`. Each `it(...)` describes one behaviour you will build. Read them before you write code: they are your specification.
3. Install the Vue DevTools browser extension and find the component tree (App, EventList, EventCard).

You need Node 24 LTS. If `npm install` fails with an error about `edgesOut`, your npm is too old: update Node.

Part B: EventCard (25 min)

Open `src/components/EventCard.vue` and complete the TODOs.

TODO	What to write	Checked by
1–2	`defineProps` with an `event` prop of type `CampusEvent`; `defineEmits` with a `book` event that sends the id	`npm run check`
3	Three `computed` values: `left`, `isFull`, `almostFull` (1 to 5 seats left)	tests
4–5	The event's category, title, date (`formatDate`) and location; class `full` on the card when full	tests
6	`v-if` / `v-else`: "Full" badge, or "N seats left" ("1 seat left" for one), class `warn` when almost	tests

TODO	What to write	Checked by
	full	
7	`:disabled` when full; `@click` emits `book` with the event id	tests

The card does not change any data itself. It reports “the user wants to book event 7” and its parent decides what that means. If you write `props.event.booked++`, Vue warns you: props are read-only.

All 5 tests in `EventCard.test.ts` pass when you are done.

Part C: EventList (25 min)

Open `src/components/EventList.vue`.

1. Declare a `book` event, the same as in EventCard, so the list can pass it up to App.
2. Add two refs: `search` (a string) and `category` (`Category | "all"`, starting at `"all"`).
3. Write a `computed` called `visible`: events of the chosen category whose title or location contains the search text, ignoring case.
4. Bind the input and the select with `v-model`.
5. Show “`visible.length` of `events.length` events”.
6. Render one `EventCard` per visible event with `v-for` and `:key="e.id"`, and pass its `book` event up. When nothing is visible, show the empty message instead (`v-if` / `v-else`).

Try it in the browser: type “lab” and you should see 2 events. Then open the DevTools component tree and watch `search` change as you type.

The filter is a `computed`, not a second `ref` you update in a `watch`. A value you can calculate from other state is always computed.

Part D: Booking in App.vue (15 min)

`App.vue` owns the events, so it is the one that changes them.

1. Add a ref `myBookings`: the ids of the events you booked, one entry per seat.
2. Add a computed `bookingCount` and show “You have booked 1 seat.” or “N seats.” in the header.
3. Write `onBook(id)`: find the event; if it still has a seat, increase its `booked` and push the id into `myBookings`.

Book seats on the Git Workshop until it is full. The card turns red at 5 seats or fewer, then shows Full, and the button is disabled. You changed one number; Vue updated the text, the colour, the badge, the button and the header.

Bonus (the last App test): keep your bookings after a page reload. `watch` `myBookings` and save it to `localStorage` with `JSON.stringify`. When the app starts, read it back and add those seats to each event’s `booked`. Wrap the read in `try` / `catch`: the stored text might not be valid JSON.

You are done when `npm test` shows 15 of 15 passing and `npm run check` shows no errors.

Part E: Start your own Vue app (45 min)

Work in your project repository, on a branch called `frontend-setup`.

1. Read your M0 feedback with the lecturer. Cut or adjust stories now if your scope is too big.
2. Draw wireframes of your 3–5 main screens (paper, Figma or Excalidraw). Save them in `docs/wireframes/`.
3. For each screen, write its component tree in `docs/components.md`: component names, their props, their events, and which component owns each piece of state.
4. Scaffold the app in a `frontend/` folder:

```
npm create vue@latest frontend
# TypeScript, Router, Pinia, Vitest, ESLint + Prettier: Yes
cd frontend && npm install && npm run dev
```

5. Add your entities to `src/types.ts`, with a few hard-coded examples in `src/data/`.
6. Build 2–3 of your screens as components on that data. Use props down and events up, as in Parts B to D.
7. Commit in small steps, push, open a pull request and merge it.

Not your project's domain: Campus Events is the class example. Your app uses the idea approved in M0.

Explain it

Answer in `week04/ANSWERS.md`, 2–4 sentences each.

1. In EventList, what goes wrong if you use the array index as `:key` and the list gets filtered while a card has local state (for example an open details panel)?
2. Why is `visible` a `computed` and not a function called from the template? What is the difference when you type in the search box?
3. EventCard emits `book` instead of changing `event.booked`. Give two reasons.
4. You write `const { booked } = reactive(event); booked++`. The page does not change. Why?
5. Where does `myBookings` live, and why not in EventCard or EventList?

Checkpoint before you leave

- ☐ `npm test` passes 15 of 15 and `npm run check` shows no errors
- ☐ You can explain props, events and computed using your own code
- ☐ Wireframes and component tree are in your project repository
- ☐ Your Vue app runs and shows 2–3 screens with hard-coded data
- ☐ `week04/ANSWERS.md` answers the five questions

Next week: routing, forms, a Pinia store and a mock API. M1 (frontend prototype) is due at the end of week 5.